UrbanMart - Documentation
Panagiotakopoulos Georgios - Capstone

NOTEBOOK 1 - BRONZE (5 cells) - raw ingestion and data profiling

This notebook copies the six CSV files into six Bronze Delta tables without changing the data, proves that every row arrived, and profiles the data quality. Bronze is my safety net: it keeps what the source really sent and I can show exactly what was wrong in the files.

Cell 2 - Create Tables

What it does: creates one Delta table per CSV (customers, customer_changes, products, stores, orders, order_items) with column comments, plus two helper tables, dq_log and bronze_file_ledger. Why every business column is STRING. I added _file_version (a fingerprint of the file) and _batch_id (which run loaded the row) so I can tell if a file changed and which run produced each row. dq_log stores every data quality finding (table, column, problem, rows affected, handling rule, time) and gets an automatic id. bronze_file_ledger is an audit trail with one row per file per run (file, version, table, status, row count, batch). I use it to prove that what was loaded equals what the file contained.

Cell 3 - load the six files

What it does: creates a variable with a new batch id (uuid), then for each file reads the CSV with read_files without type inference, adds the lineage columns and writes it with INSERT ... REPLACE WHERE on the file name. After each load it writes one ledger row with the number of rows of that batch. Why REPLACE WHERE: a plain INSERT would add the file again every time I rerun the notebook. REPLACE WHERE first removes the rows that came from the same file and then inserts them, so running the notebook once or many times gives the same table (idempotent).

Cell 4 - Bronze validation

What it does: one row per table. For each table it counts the rows directly in the CSV, counts the rows in the Bronze table and reads the latest SUCCESS row count from the ledger, and compares them with assert_true. It also checks that the file is not empty, that no row ended up in the rescued-data column, that only one file version and one batch exist in the table, and that no lineage column is NULL.

Cell 5 - data quality profiling (one large cell)

What it does: empties dq_log (TRUNCATE, so each run describes the current data) and then fills it step by step. Every handling rule starts with one keyword (DEDUPLICATE, STANDARDISE, DEFAULT, REJECT or REVIEW) so it can be parsed and counted.

- Step 1, exact duplicates: groups each table by all business columns and counts the extra copies.
- Step 2, missing values that can be defaulted. (gender for example)
- Step 3, missing values that force a rejection. (keys for example)
- Step 4, invalid types: dates, store size and discount that are not blank but fail try_to_date, try_to_timestamp or try_cast.
- Step 5, invalid types (reject). (string quantities for example)
- Step 6, conflicting duplicate keys: the same id with different content, compared as JSON so a NULL against a value also counts. row_affected counts all rows in the conflicting groups. I cannot know which

version is correct, so these are reviewed or rejected rather than guessed.
- Step 7, logical errors: future dates, negative costs, prices, sizes and discounts, quantity below 1, list price below cost, invalid active flag, invalid email, signup before birth, orders before store opening or customer signup, payment method and order status that differ only by case or are not allowed, orders pointing to a customer or store that does not exist, order lines pointing to an order or product that does not exist, customer changes for a missing customer, and discounts that look like percentages (1 to 100) or are above 100.
- Step 8, summary: two queries, the first counts checks per action keyword, the second lists only the checks that found something. These are the tables I use in the documentation. Note: dq_log holds the recommended rule at profiling time The rule that is really applied is the one implemented in Silver. Where Silver defaults a value (for example an unknown payment method becomes Unknown) the Silver notebook is the final decision.

NOTEBOOK 2 - SILVER (9 cells) - cleansing, typing and rejects

This notebook reads the Bronze tables and builds clean, properly typed tables. Every Bronze row ends up at the Silver table (good row) or silver.etl_rejects (refused row with a reason code). Pattern for every table: build a staging table that holds the cleaned values and a reject code per row, write the good rows to the Silver table with INSERT OVERWRITE, and replace that table's rows in etl_rejects.

Cell 2 - table definitions

What it does: defines the six cleaned tables with real types (DATE, INT, DECIMAL(12,2), DECIMAL(5,4), BOOLEAN) and NOT NULL on the columns that must always have a value, plus etl_rejects.

Cell 3 - customers

Cleaning: trim and INITCAP names, city, region and segment (blank becomes Unknown), email is lower-cased and kept only if it matches a basic email pattern, otherwise NULL. Gender is mapped to F, M or Unknown, birth and signup dates are parsed with try_to_date so a bad value becomes NULL. Duplicates: the same customer_id with different content is a CONFLICT_KEY and all its rows are rejected. An identical copy keeps one row and rejects the extras as DUP_EXACT. Rejects: missing customer_id and a birth date in the future.

Cell 4 - customer_changes

These rows say that a customer's city, region or segment changed on an effective date. Rejects: missing customer_id or effective_date, an effective date that cannot be parsed, the same customer and date with different content, exact duplicates, and changes for a customer that does not exist in Silver customers (ORPHAN_FK).

Cell 5 - products

Cleaning: product name stays as it is (Unknown if blank), category, subcategory and brand are trimmed and INITCAP, the active flag Y/N becomes a BOOLEAN, cost and price are converted to DECIMAL(12,2). Rejects: missing product_id, cost or price missing, not a number, negative or with more than two decimals, conflicting or duplicate product ids.

Cell 6 - stores

Cleaning: text trimmed and INITCAP, blank becomes Unknown. An opening date in the future or a negative size is wrong but not fatal, so only that value becomes NULL and the store is kept. Rejects: missing store_id, conflicting content for the same store_id, exact duplicates. Why keep the store: orders refer to stores, so rejecting a store would remove its sales from the regional analysis.

Cell 7 - orders

Cleaning: the order timestamp is checked with a pattern (date with optional time) and the date part is parsed with try_to_date, so a malformed value becomes NULL, a date in the future also becomes NULL (Gold maps a NULL date to the unknown date -1). Payment method is trimmed and anything other than Card, Cash or Mobile Wallet becomes Unknown. Order status is trimmed, rejects: missing order_id, a missing or invalid status (must be Completed, Cancelled or Returned), conflicting or duplicate orders. Orders that point to a customer or store that does not exist are NOT rejected: I keep customer_id and store_id as received and Gold maps them to -1. The order really happened and has value, so losing it would understate revenue. Why the status is mandatory: revenue and return analysis depend on it.

Cell 8 - order_items

This is the most important Silver table: every row becomes one row of the fact table. Cleaning: quantity, unit price and discount are converted to numbers. The discount becomes a fraction: blank = 0, between 0 and 1 kept, between 1 and 100 treated as a percentage and divided by 100. Anything else is rejected. Rejects: missing line, order or product id, quantity missing, invalid or below 1, price missing, invalid, negative or with more than two decimals, invalid or out-of-range discount, conflicting or duplicate lines, and lines whose order or product does not exist in the Silver orders or products tables (ORPHAN_FK). Without a product there is no cost and no margin. Without an order there is no date, store or customer. The reject checks are evaluated in this order: missing key, invalid value, conflict, duplicate, orphan.

Cell 9 - reconciliation

For each table it checks with assert_true that Bronze rows = Silver rows + reject rows and that the key is unique in Silver (for customer_changes the key is customer plus effective date). A second query checks that every Silver order line has an order and a product in Silver and that every reject has a reject code. Why: this proves that no row was lost or invented and is the foundation of the row-count test in the validation notebook. A failure stops the notebook and the job.

NOTEBOOK 3 - GOLD - star schema

This notebook builds the data mart: dim_date, dim_customer, dim_product, dim_store and fact_sales. Kimball's four steps: business process = sales, grain = one row per order line, dimensions = date, customer, product, store and facts = quantity, gross, discount, net, cost and margin. A star schema makes questions like "revenue by month and region" simple joins and fast scans.

Cell 2 - dimension tables

What it does: creates dim_date, dim_customer, dim_product and dim_store with CREATE TABLE IF NOT EXISTS. Surrogate keys are BIGINT GENERATED BY DEFAULT AS IDENTITY: the warehouse owns its keys and is independent of source ids. "By default" lets me insert the fixed unknown member -1 and the yyyymmdd date keys myself. Each table has a primary key. In dim_date I added weekend flag and day type because the fifth business question compares weekdays with weekends. dim_customer, dim_product and dim_store keep the source id as the natural key, dim_store has is_online and channel because the source has no online/physical column.

Cell 3 - fact table and constraints

What it does: creates the fact table with the grain key order_line_id, the order id (both degenerate dimensions), four foreign keys, order status and payment method, the original quantity, price, discount and cost, and the five amounts (gross, discount, net, cost, margin). Primary and foreign keys document the model and drive the ERD. CLUSTER BY (date_key) is liquid clustering, because most questions filter or group by time. CHECK constraints are added with ALTER TABLE because CREATE TABLE does not

accept them here. They are enforced by Delta: quantity at least 1, no negative price or cost, discount between 0 and 1, discount not larger than gross, and the equations gross = quantity x price, net = gross - discount and margin = net - cost, plus a valid order status. Each constraint is dropped first (DROP CONSTRAINT IF EXISTS) so the cell can be rerun without an "already exists" error.

Cell 4 - dim_date

Generated with sequence() from 1 January of the first order year to 31 December of the last order year, so every day exists even when no order was placed. Only dates that are not already present are inserted, and the unknown date -1 is added once, so a rerun does not duplicate anything.

Cell 5 - dim_customer

Loaded with MERGE on customer_id. Customer changes: for each customer I take the latest change dated today or earlier, attribute by attribute (a NULL means unchanged), and use it instead of the original city, region or segment. The unknown customer (-1, "Unknown") is inserted once and receives orders whose customer is missing or does not exist.

Cell 6 - dim_product

Same MERGE pattern on product_id with category, subcategory, brand, cost and list price. The product cost is used to calculate the cost and margin of each sale. The unknown product (-1) is inserted once to complete the model.

Cell 7 - dim_store

Same MERGE pattern on store_id. is_online and channel (Online / Physical / Unknown) are derived from the store type with a text pattern because the source has no such column. The unknown store (-1) is inserted once.

Cell 8 - fact_sales load

What it does: one query reads the Silver order lines, joins their order header and looks up the surrogate keys with LEFT JOINs to the four dimensions. A missing match becomes -1 (COALESCE), so no sales row is ever dropped by a join.

Cell 9 - OPTIMIZE and counts

OPTIMIZE compacts the files and applies the liquid clustering. The query lists the row count of each Gold table. Every dimension has one more row than its Silver table: the unknown member.

Analytics cells (the five queries)

Only Gold tables are used, as the assignment requires. Rules for all queries: revenue is the net amount of Completed orders and Cancelled orders are excluded, margin percentage is total margin divided by total revenue (a ratio of sums).

Q1 Monthly sales performance

Revenue, gross margin, margin % and units sold by month, region and store, sorted by month and revenue. Answers: where and when do we sell and earn?

Q2 Product performance

Revenue, margin and margin % by category and brand, ranked by margin with RANK. Answers: which products drive profit, not just revenue?

Q3 Customer performance

Top 5 customers per segment: revenue, margin and number of orders per customer, ROW_NUMBER over each segment ordered by revenue. The unknown customer (-1) is excluded because it is not a real customer.

Q4 Returns analysis

Return rate by category and store type. Cancelled lines are excluded because they never became a sale. I show a rate by lines (returned lines divided by valid lines) and a rate by value (returned net divided by total net) because many cheap returns and a few expensive returns tell different stories.

Q5 Sales patterns

Weekday versus weekend and online versus physical: revenue, orders, average order value, active days, revenue per day and margin %. Revenue per day makes the comparison fair, because there are more weekdays than weekend days and the totals alone would favour weekdays. Rows with the unknown date are excluded.

NOTEBOOK 4 - VALIDATION (7 cells)

Every check uses assert_true, so a wrong result makes the notebook fail with a message naming the check and the numbers. It is the last task of the Lakeflow Job, so wrong data turns the whole run red instead of reaching the dashboard. This matters because Unity Catalog does not enforce primary or foreign keys, and a pipeline can finish "successfully" while producing wrong data.

Cell 2 - Test 1 (row reconciliation from Bronze to Gold)

Bronze order-item rows minus the order-item rows in etl_rejects must equal the rows in fact_sales. Both sides count physical rows, so they are at the same grain. The message shows all the numbers when it fails.

Cell 3 - Test 1b (grain)

fact_sales has one row per order_line_id (rows = distinct order_line_id), and the number of Silver order lines equals the number of fact rows. This proves that no line was duplicated or dropped between Silver and Gold.

Cell 4 - Test 2 (totals)

The total net amount in Silver, calculated again from quantity, unit price and discount with the same rounding, must equal the total net amount in Gold. Equal row counts do not prove equal values. This proves that no amount was changed, lost or counted twice.

Cell 5 - Test 3 (orphans)

For date, customer, product and store, a LEFT ANTI JOIN from the fact to the dimension must return 0 rows. The foreign key constraints are only documentation, so this is what really protects the star schema.

Cell 6 - Test 4 (primary keys)

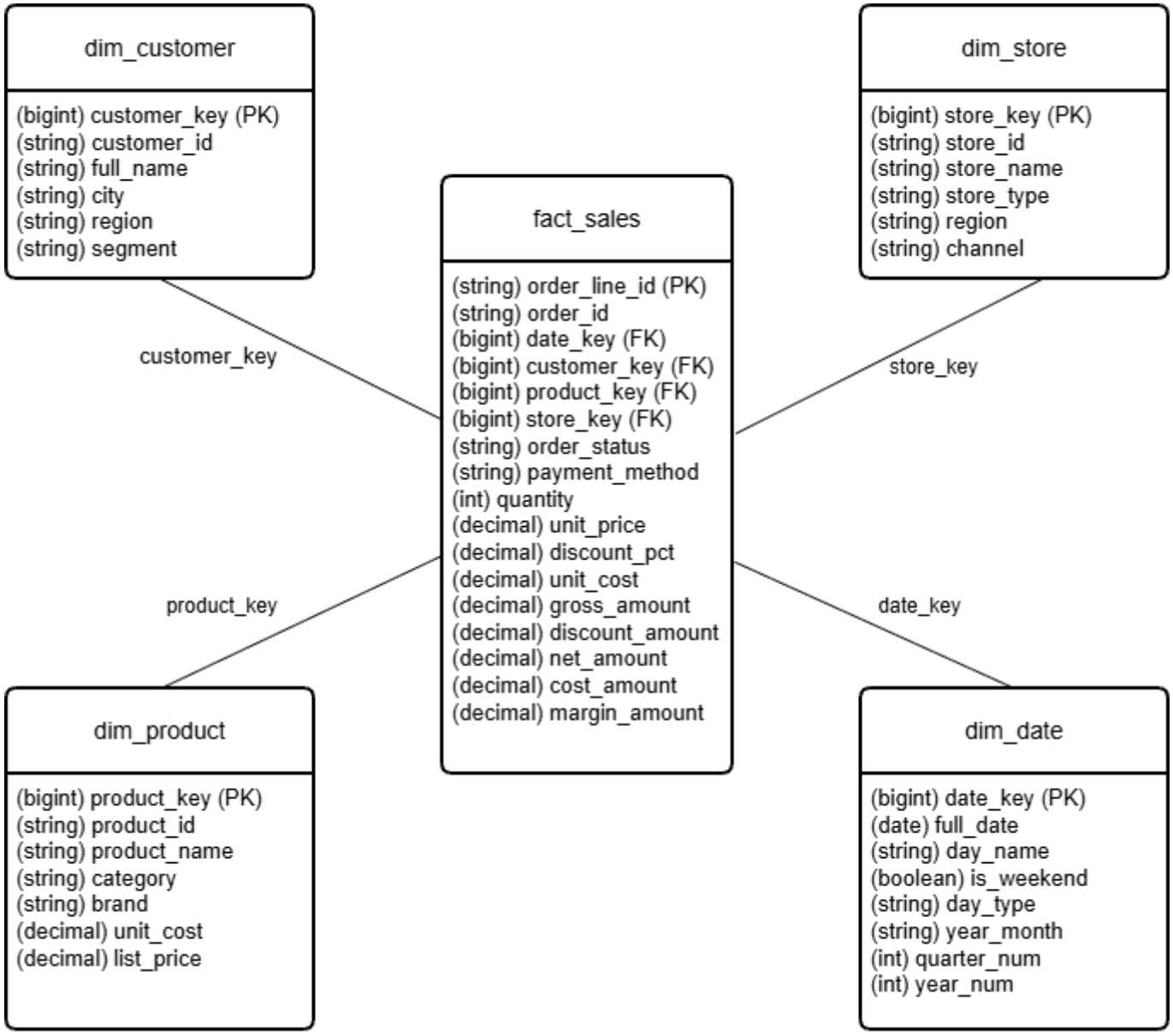
For the four dimensions and the fact, rows minus distinct keys must be 0 and there must be no NULL key. The natural keys of customer, product and store must also be unique.

Cell 7 - Test 5 (extras)

Each dimension has exactly one unknown member (-1 customer), product and store dimensions have the same number of rows as Silver (plus the unknown member) and total margin equals total net minus total cost.

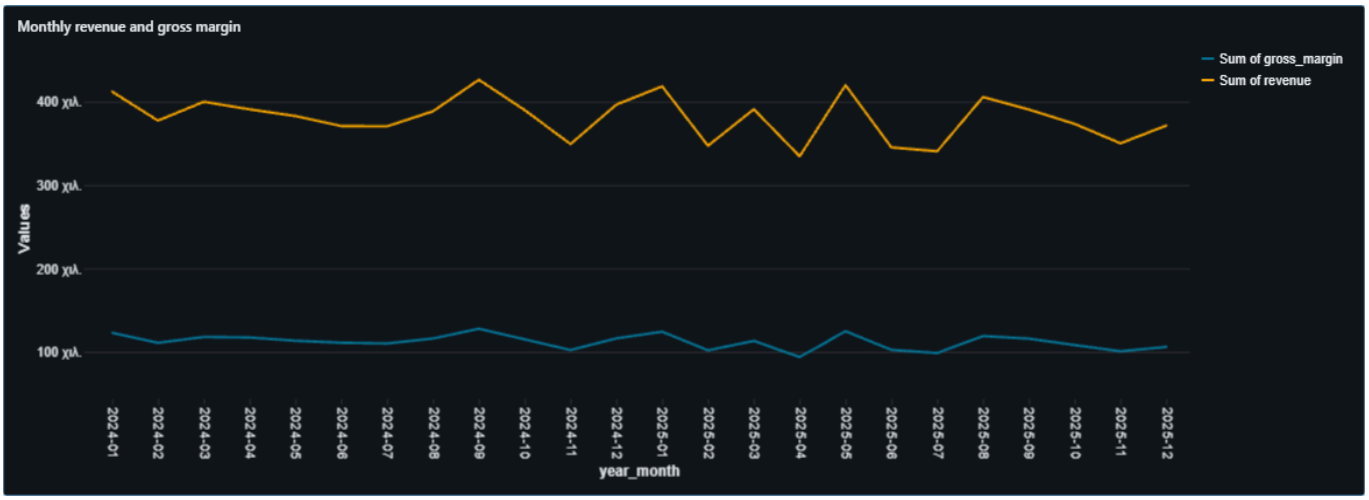
ERD

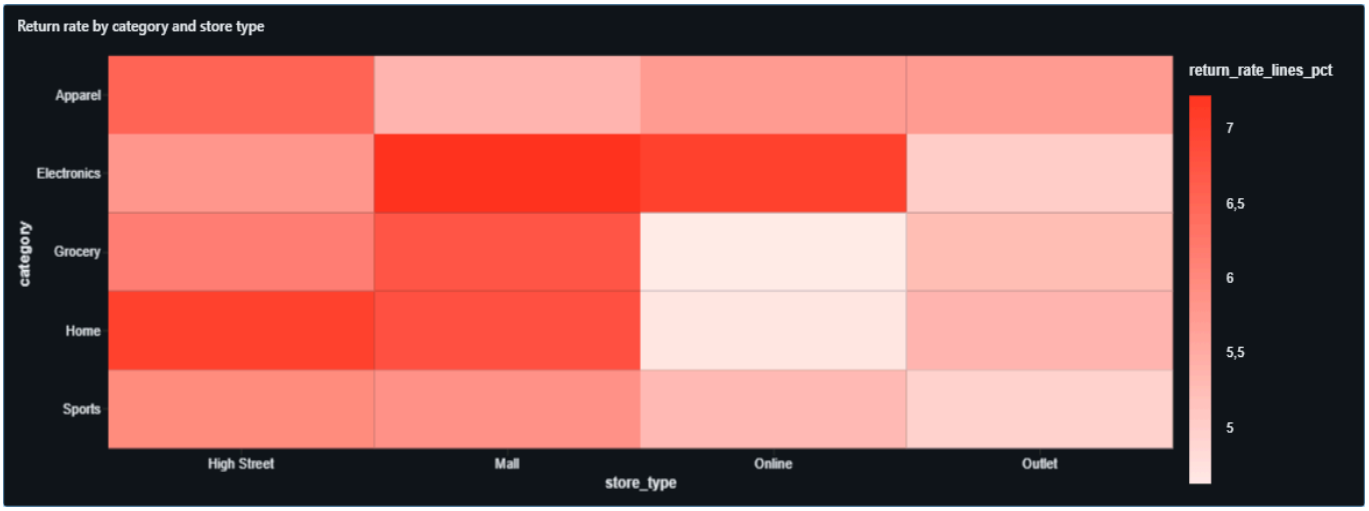
The Gold layer is a star schema. fact_sales is in the centre and holds one row per order line. It is linked to four dimensions: dim_date, dim_customer, dim_product and dim_store, each through its surrogate key (date_key, customer_key, product_key, store_key). Each link is one-to-many: one dimension row can appear in many sales lines, but each sales line points to exactly one dimension row.



Dashboard

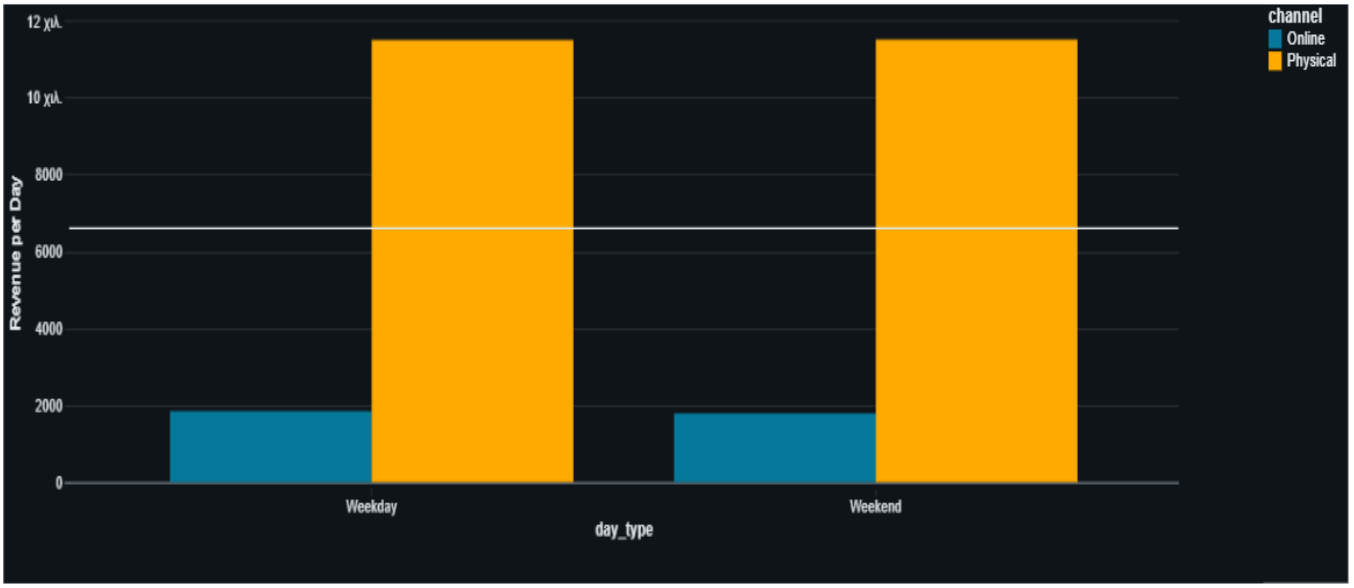
The dashboard (*UrbanMart Sales Dashboard*) uses only Gold tables. Each visual has its own SQL dataset, and revenue always means the net amount of Completed orders. Percentages are calculated as a ratio of sums, never as an average of percentages.







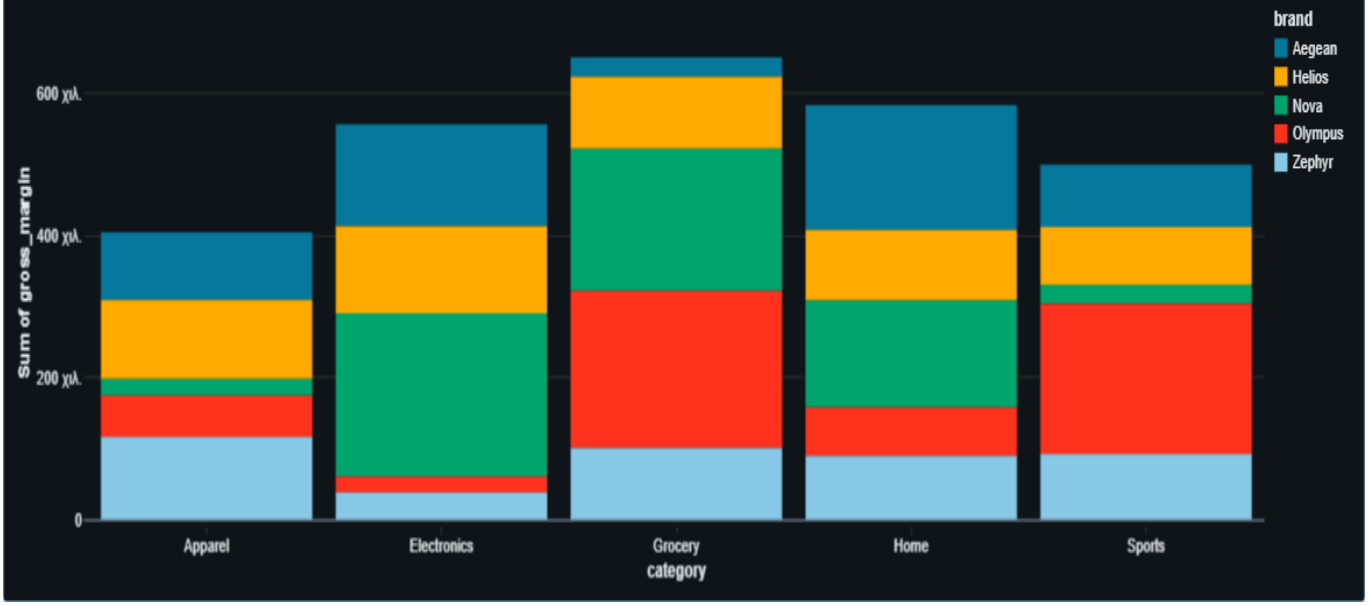
Revenue per day: weekday vs weekend, online vs physical



Top 5 customers by segment

segment	customer_rank	customer_id	full_name	revenue	gross_margin
Loyalty Gold	1	C0065	Giorgos Pappas	37217,82	11459,82
Loyalty Gold	2	C0345	Sofia Ioannou	33204,49	8988,41
Loyalty Gold	3	C0073	Nikos Georgiou	31291,02	9705,86
Loyalty Gold	4	C0225	Dimitris Ioannou	28942,73	8107,10
Loyalty Gold	5	C0126	Maria Vasileiou	28246,13	8022,66
Loyalty Silver	1	C0043	Dimitris Oikonomou	32752,52	9813,76
Loyalty Silver	2	C0462	Nikos Vasileiou	29614,59	9007,21
Loyalty Silver	3	C0411	Sofia Georgiou	28718,72	8432,86
Loyalty Silver	4	C0104	Maria Ioannou	28651,28	9194,71
Loyalty Silver	5	C0422	Nikos Georgiou	27970,37	8987,75
Retail	1	C0288	Giorgos Papadopoulos	36810,63	10914,40
Retail	2	C0286	Ioanna Christodoulou	31075,97	8704,02
Retail	3	C0218	Katerina Nikolaou	30885,35	9531,34
Retail	4	C0378	Giorgos Oikonomou	29566,23	9162,60
Retail	5	C0145	Eleni Nikolaou	29251,63	8586,81

Gross margin by category and brand



Orchestration

The pipeline is orchestrated by a Lakeflow Job, *urbanmart_pipeline*, with four notebook tasks on serverless compute: Bronze, Silver, Gold and Validation. Each task depends on the previous one, so a failure stops everything after it, and Validation runs last as the final success/failure gate.

